

Database Management System

Semester-III (Batch-2024)

Collaborative Project Management



Supervised By:
Dr. Amanpreet Kaur

Submitted By:
Vaibhav Katyal , 2410990480
Madhav Gautam , 2410990429
Medhansh Garg , 2410990452

**Department of Computer Science and Engineering
Chitkara University Institute of Engineering & Technology,
Chitkara University, Punjab**

Table of Contents

S.No.	Practical
1	Introduction
2	E-R Diagram
3	Normalization Used
4	Database Schema
5	References

Introduction

The Collaborative Project Management System is a role-based application designed to simplify and streamline teamwork on projects. It provides a structured platform where users can work together, assign tasks, track progress, and evaluate performance effectively. The system ensures that every project member—from team leaders to contributors—has defined roles, responsibilities, and access levels, which helps in creating a transparent and organized workflow.

In today's world, where teamwork and collaboration form the backbone of successful projects, it becomes essential to have a system that centralizes project details, manages tasks, and facilitates smooth communication. Unlike other platforms, there is currently no system that truly helps in judging a person's **leadership qualities** or **teamwork skills**—this system is designed to fill that gap. Along with projects and tasks, it also introduces a **community feature** where users can directly share their projects, let others view them, and receive ratings and constructive feedback.

The Collaborative Project Management System is designed with a well-structured database that eliminates redundancy, ensures data integrity, and supports efficient data retrieval. It captures every detail of projects, members, tasks, subtasks, ratings, and badges in a way that ensures accountability and promotes motivation among team members.

The purpose of this system is to create a **centralized collaborative platform** that enables:

- Users to join projects and contribute through assigned tasks and subtasks.
- Leaders to assign responsibilities, manage deadlines, and track progress.
- Teams to evaluate each other's performance through ratings and constructive feedback.
- The organization of achievements through points, badges, and streaks to encourage consistent performance.

By combining features of task management, performance monitoring, and motivational rewards, the system not only helps in completing projects on time but also fosters a culture of teamwork and accountability.

System Features and Role-based Functions

User Functions

- Create and manage personal profiles with details like bio, skills, and interests.
- Join different projects as team members or leaders.
- Receive and accept team match requests to collaborate with others.
- Work on assigned tasks and subtasks, while updating status and progress.
- Maintain streaks to stay consistent in contributions.
- Earn badges and points for completing milestones and achieving targets.

Leader Functions

- Create and manage projects by defining scope, deadlines, and requirements.
- Assign tasks and subtasks to team members according to skills.
- Monitor project progress and track individual contributions.
- Provide feedback and ratings to team members for their performance.
- Oversee the distribution of badges and motivate team members.

Project Functions

- Allow multiple members to collaborate on a single project.
- Provide structured task management with deadlines and progress tracking.
- Enable rating and feedback features for fair evaluation.
- Maintain records of completed projects, member contributions, and outcomes.

System Functions

- Ensure secure storage and retrieval of project-related data.
- Manage relationships between users, tasks, projects, ratings, and badges.
- Maintain leaderboards, streaks, and achievements for transparency and motivation.
- Support real-time collaboration through requests and notifications.

Conclusion

The Collaborative Project Management System is not just a task management tool; it is a complete platform to encourage teamwork, improve accountability, and enhance efficiency. With features like task assignment, feedback-based ratings, streak tracking, and badge-based motivation, it goes beyond traditional management systems by combining productivity with engagement.

By implementing a carefully designed database structure, this system ensures consistency, reduces redundancy, and maintains high data integrity. Ultimately, it empowers individuals and teams to achieve project goals effectively while building a culture of collaboration and recognition.

E-R Diagram

An Entity–Relationship (E-R) Diagram is a conceptual model that describes the structure of a database in terms of **entities, attributes, and relationships**. It provides a high-level view of how data is stored, processed, and connected in the system. The E-R diagram is widely used in database design because it gives a **clear and visual representation of the system requirements** before the actual implementation of the database.

In our project, **Collaborative Project Management System**, the E-R diagram represents how users, projects, tasks, ratings, badges, and other entities interact with each other to form a structured and efficient collaboration platform.

Key Components of our E-R Diagram

1. Entities

Entities represent real-world objects or concepts that store information in the database. They are drawn as **rectangles** in the diagram.

Examples in our project:

- **Users** – stores user details like profile, skills, and contact info.
- **Projects** – stores project-related information.
- **Tasks** – represents tasks created under projects.
- **Subtasks** – smaller units of tasks to track progress in detail.
- **Ratings** – feedback and performance evaluation.
- **Badges** – motivational rewards assigned to users.
- **Team Match Request** – requests to join or collaborate in teams.
- **Project Members** – represents members assigned to a project.

2. Attributes

Attributes describe the properties or characteristics of an entity. They are represented as **ellipses (ovals)** connected to their entity.

Examples in our project:

- **Users:** user_id, full_name, email, bio, profile_pic, skills, points.
- **Projects:** project_id, title, description, status, deadline.
- **Tasks:** task_id, title, description, deadline, status.
- **Subtasks:** subtask_id, subtask_title, deadline, status.
- **Ratings:** rating_id, teamwork, communication, feedback.
- **Badges:** badge_id, badge_name, description, points_required.

3. Primary Key

A **primary key** is a unique attribute that identifies each record of an entity.

Examples in our project:

- user_id uniquely identifies a User.
- project_id uniquely identifies a Project.
- task_id uniquely identifies a Task.
- rating_id uniquely identifies a Rating.

4. Relationships

Relationships represent the associations between entities and are shown as **diamonds** in the diagram.

Examples in our project:

- A **User** is *Assigned* to many **Tasks**.
- A **Project** *Has* many **Tasks**.
- A **Task** *Has* multiple **Subtasks**.
- A **User** can send a **Team Match Request** to join a **Project**.
- A **User** *Receives* Ratings and can *Earn* Badges.
- A **Project** *Has* Rating given by users.

5. Cardinality

Cardinality defines how many instances of one entity are related to another entity.

Examples in our project:

- One **User** can be a member of many **Projects** (1:M).
- One **Project** can have many **Tasks** (1:M).
- One **Task** can have many **Subtasks** (1:M).
- Many **Users** can collaborate on many **Projects** (M:N).

Importance of E-R Diagram

- Provides a clear **visualization** of users, projects, and task relationships.
- Ensures **data consistency** and avoids redundancy.
- Helps developers and stakeholders understand the **system structure** before database implementation.
- Forms the **foundation for relational database design** by mapping entities to tables.
- Supports better project planning and collaboration through a structured model.

Thus, the E-R diagram provides a **complete overview** of how data is structured, stored, and linked in the **Collaborative Project Management System**, ensuring smooth collaboration, efficient task tracking, and proper evaluation of performance.

Main Entities in Collaborative Project Management System

An **Entity** in a database represents a real-world object, concept, or person about which data needs to be stored. Entities form the **building blocks** of an E-R diagram and are usually represented as **rectangles**. Each entity is described by its **attributes**, and every entity has a **primary key** that uniquely identifies its records.

Entities are of two types:

1. **Strong Entity** – Exists independently and has its own primary key.
Example: User, Project, Task.
2. **Weak Entity** – Depends on another entity for its existence and does not have a primary key of its own. *Example: Subtask (depends on Task), Project Members (depends on Project and User).*

Main Entities in our Project

1. **User** – Represents individuals using the system. Each user has attributes such as `user_id`, name, email, bio, profile picture, skills, and points. Users can be members of projects, perform tasks, and receive ratings or badges.
2. **Project** – Represents collaborative projects created and managed by users. Attributes include `project_id`, title, description, status, and deadline. A project can have multiple members and tasks.
3. **Project Members** – A weak entity that represents the association of users with projects. It stores details like role (leader/member) and links users to their respective projects.
4. **Task** – Represents tasks created under projects. Attributes include `task_id`, title, description, deadline, and status. Tasks are assigned to users for execution.
5. **Subtask** – A weak entity under Task that breaks down work into smaller units. Attributes include `subtask_id`, title, status, and deadline.
6. **Team Match Request** – Stores requests between users to join or collaborate in projects. Attributes include `request_id`, status, and timestamps.
7. **Rating** – Stores performance evaluations and feedback. Attributes include `rating_id`, teamwork, communication, feedback, and `created_by`. Ratings help in measuring contribution quality.
8. **Badge** – Represents rewards or achievements earned by users. Attributes include `badge_id`, `badge_name`, description, and `points_required`.
9. **User Badges** – A weak entity that links users with the badges they have earned. It records details like `earned_at` and `user_id`.

10. **Task Streaks** – Tracks consistency of users in completing tasks. Attributes include streak_id, count, and last_updated.
11. **Contribution Badges** – Stores contribution-specific rewards given to users for teamwork and collaboration.

Thus, the **main entities** of the Collaborative Project Management System collectively support the functionality of project creation, team collaboration, task management, performance evaluation, and motivation through ratings and badges.

Main Attributes in Collaborative Project Management System

An **Attribute** is a property or characteristic that describes an entity in a database. Attributes define the **details stored** about each instance of an entity. In an E-R diagram, attributes are represented by **ellipses (ovals)** connected to their respective entities.

Types of Attributes

1. **Simple Attribute** – Cannot be divided further. *Example: Name, Email.*
2. **Composite Attribute** – Can be divided into smaller parts. *Example: Full_Name → First_Name + Last_Name.*
3. **Derived Attribute** – Obtained from other attributes. *Example: Task Streak Count (derived from completed tasks over time).*
4. **Multi-valued Attribute** – Can have multiple values for a single entity. *Example: Skills for a User.*
5. **Key Attribute** – Used to uniquely identify an entity (Primary Key). *Example: user_id, project_id.*

Attributes of Main Entities in our Project

1. **User**
 - user_id (PK), first_name, last_name, email, bio, profile_pic, skills, points, join_date, password, leadership_points, created_at
2. **Project**
 - project_id (PK), project_name, description, status, deadline, created_at, updated_at, tech_stack, workspace, closed_at, leader_id (FK)

3. Project Members

- team_id (PK), user_id (FK), project_id (FK), role_in_project ,contribution_points.

4. Task

- task_id (PK), project_id (FK), t a s k _title, description, deadline, status, priority, created_at, assigned_to (FK), kanban_column

5. Subtask

- subtask_id (PK), task_id (FK), subtask_title, status.

6. Team Match Request

- request_id (PK), user_id (FK), team_id (FK), status, requested_role.

7. Rating

- rating_id (PK), project_id (FK), rated_user (FK), teamwork, communication, feedback, created_at, rated_by.

8. Badge

- badge_id (PK), badge_name, description, points_required.

9. User Badges (*Weak Entity*)

- user_id (FK), badge_id (FK), earned_at.

10. Task Streaks

- streak_id (PK), user_id (FK), count, last_completed, longest_streak, current_streak

11. Contribution Heatmap

- heatmap_id (PK), user_id (FK), contribution, date.

These attributes collectively describe the **data structure** of the Collaborative Project Management System. They ensure that users, projects, tasks, ratings, and achievements are stored in a **well-defined, non-redundant, and relational format**, supporting smooth project execution and collaboration.

Main Relationships & Cardinality in Collaborative Project Management System

Relationships

A **Relationship** in an E-R diagram represents the **association or interaction** between two or more entities. It shows how entities are connected and how data flows between them. Relationships are represented as **diamonds** connecting entities in the diagram.

Types of Relationships

1. **One-to-One (1:1):** One instance of an entity is associated with exactly one instance of another entity.
2. **One-to-Many (1:N):** One instance of an entity is associated with multiple instances of another entity.
3. **Many-to-Many (M:N):** Multiple instances of one entity are associated with multiple instances of another entity.

Cardinality in our System

1. **User → Project Members**
 - **Type:** One-to-Many (1:M)
 - **Explanation:** One user can be a member of many projects.
 - **Key Link:** User.user_id (PK) → ProjectMembers.user_id (FK)
2. **Project → Project Members**
 - **Type:** One-to-Many (1:M)
 - **Explanation:** One project can have many users as members.
 - **Key Link:** Project.project_id (PK) → ProjectMembers.project_id (FK)
3. **User → Task**
 - **Type:** One-to-Many (1:M)
 - **Explanation:** A single user can be assigned multiple tasks.
 - **Key Link:** User.user_id (PK) → Task.assigned_to (FK)
4. **Project → Task**
 - **Type:** One-to-Many (1:M)

- **Explanation:** One project can contain multiple tasks.
- **Key Link:** Project.project_id (PK) → Task.project_id (FK)

5. **Task → Subtask**

- **Type:** One-to-Many (1:M)
- **Explanation:** Each task can be broken down into multiple subtasks.
- **Key Link:** Task.task_id (PK) → Subtask.task_id (FK)

6. **User → Team Match Request**

- **Type:** One-to-Many (1:M)
- **Explanation:** One user can send many team match requests to join projects.
- **Key Link:** User.user_id (PK) → TeamMatchRequest.user_id (FK)

7. **Project Members → Team Match Request**

- **Type:** One-to-Many (1:M)
- **Explanation:** A single project can receive multiple team match requests.
- **Key Link:** Project.project_id (PK) → TeamMatchRequest.project_id (FK)

8. **User → Ratings (Giver and Receiver both)**

- **Type:** One-to-Many (1:M)
- **Explanation:** A user can receive multiple ratings and feedback from different projects.
- **Key Link:** User.user_id (PK) → Ratings.rated_user (FK)

9. **Project → Ratings**

- **Type:** One-to-Many (1:M)
- **Explanation:** Each project can have multiple ratings given by members.
- **Key Link:** Project.project_id (PK) → Ratings.project_id (FK)

10. **User → User Badges**

- **Type:** One-to-Many (1:M)
- **Explanation:** A single user can earn multiple badges.
- **Key Link:** User.user_id (PK) → UserBadges.user_id (FK)

11. Badge → User Badges

- **Type:** One-to-Many (M:M)
- **Explanation:** Each badge can be earned by many different users.
- **Key Link:** Badge.badge_id (PK) → UserBadges.badge_id (FK)

12. User → Task Streaks

- **Type:** One-to-One (1:1)
- **Explanation:** Each user maintains one streak record that tracks consistency.
- **Key Link:** User.user_id (PK) → TaskStreaks.user_id (FK)

13. User ↔ Contribution Heatmap

- **Type:** One-to-Many (1:M)
- **Explanation:** A user can have multiple contribution records in the heatmap, but each contribution record belongs to a single user.
- **Key Links:** User.user_id (PK) → ContributionHeatmap.user_id (FK)

14. User → Projects

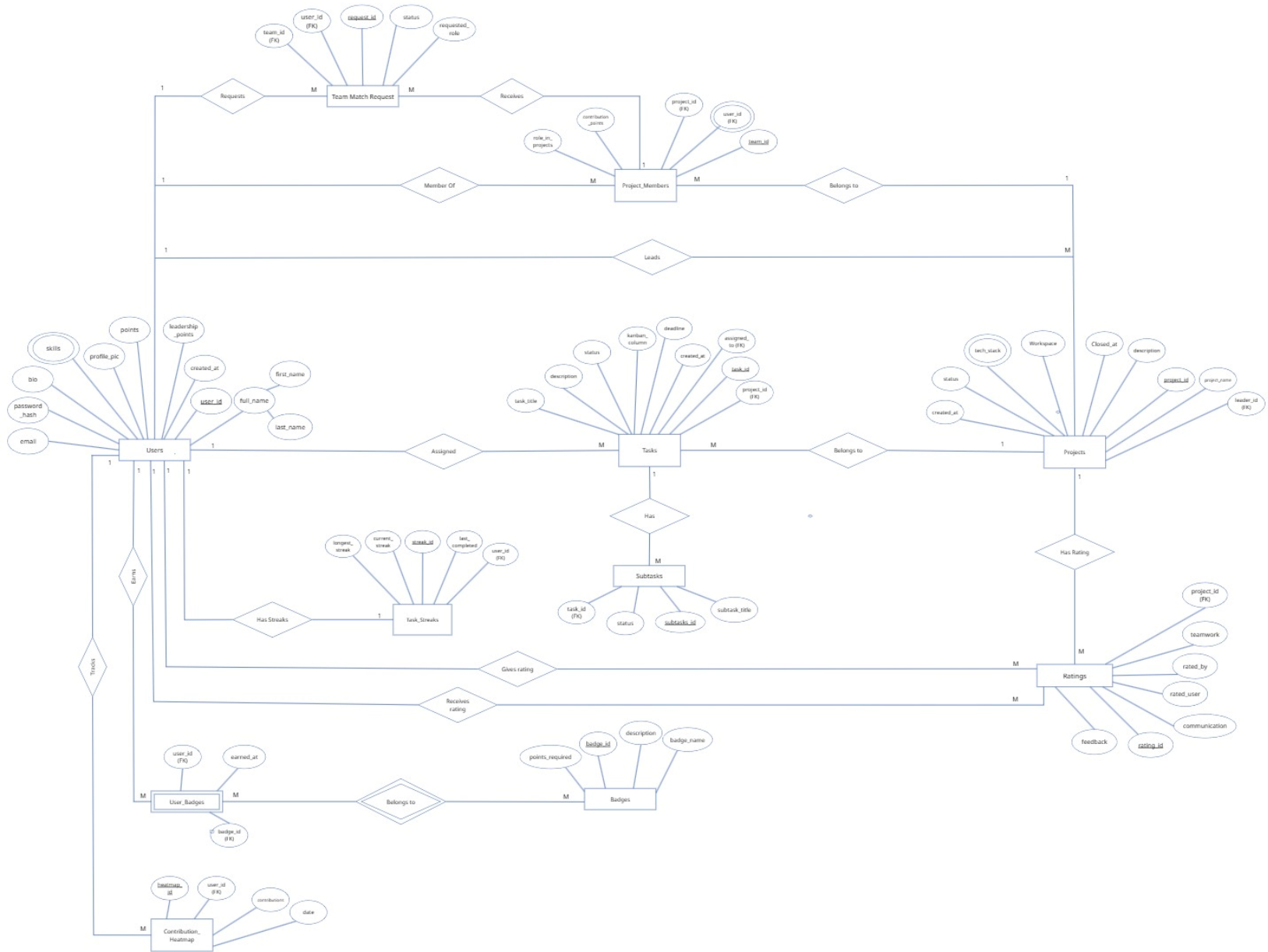
- **Type:** One-to-Many (1:M)
- **Explanation:** A single user can be lead of Multiple Projects.
- **Key Link:** User.user_id (PK) → Projects.leader_id (FK)

Importance of Relationships & Cardinality

- They define how entities in the system interact with one another.
- Help ensure **data consistency** by properly linking users, tasks, projects, and feedback.
- Avoid redundancy by modeling weak entities like **Project Members** and **User Badges**.
- Provide a **clear structure** for task assignment, collaboration, and evaluation.
- Form the basis for relational database design (foreign key mapping).

Thus, the **relationships and cardinality** in the Collaborative Project Management System capture the full dynamics of teamwork — from users joining projects, to task assignments, feedback sharing, and recognition through badges.

E-R Diagram



Link for E-R Diagram:

https://miro.com/app/board/uXjVJQxGOBE/?share_link_id=131014497847

Normalization Used

Normalization is a systematic approach to organizing data in a relational database to eliminate redundancy and prevent anomalies during insertion, update, and deletion operations. The Project Management System database has been normalized through successive stages: **First Normal Form (1NF)**, **Second Normal Form (2NF)**, **Third Normal Form (3NF)**, and **Boyce-Codd Normal Form (BCNF)**. The following section illustrates the normalization process using representative entities and attributes from the system.

3.1 Functional Dependencies

The Project Management System database contains several functional dependencies that influence how data is structured and related across entities. These dependencies define how one attribute determines another and are critical for ensuring consistency and eliminating redundancy.

3.1.1 Partial Dependencies

Partial dependencies occur when a non-key attribute depends on only part of a composite primary key. In this schema:

- User_ID determines Full_Name and Email, meaning each user has a unique identity and contact information.
- Project_ID determines Project_Name, indicating that each project has a distinct name associated with its identifier.
- Task_ID determines Task_Title and Description, showing that each task carries its own title and descriptive details.

3.1.2 Transitive Dependencies

Transitive dependencies exist when a non-key attribute depends on another non-key attribute, which in turn depends on the primary key. In this schema:

- Role_ID determines Role_Name, and User_ID is associated with Role_ID, creating an indirect dependency between User_ID and Role_Name.
- assigned_to in the Tasks entity refers to a user, and that user's details are stored separately, forming a chain of dependency.
- rated_by and rated_user in the Ratings entity reference users, whose attributes are maintained in the Users table, establishing a transitive link between ratings and user details.

These dependencies ensure that descriptive attributes are not redundantly stored and are accessed through their respective identifiers.

Non-Trivial Dependency on Non-Candidate Key

A non-trivial dependency on a non-candidate key occurs when an attribute depends on a field that is not a candidate key. In this schema:

- User_ID determines Leadership_Role, even though the composite key in the Project_Members table is (User_ID, Project_ID). This means a user's leadership designation is independent of their specific project membership and must be treated as a standalone attribute.

This dependency is handled by isolating leadership-related data to ensure it is not improperly tied to composite relationships.

3.2 Unnormalized Form (UNF)

In the initial design, multiple attributes contain composite or multi-valued data.

- A single record may contain multiple badges earned by a user
- Multiple subtasks under a task
- Multiple roles assigned to project members
- Multiple ratings given or received by users

User_ID	Full_Name	Email	Project_Name	Role_in_projects	Task_Name	Subtasks	Badges_Earned	Rated_By	Feedback
U001	Raj	raj@email.com	AlphaBuild	Developer	UI Design	Wireframe, Mockup	Leader, Contributor	U002	Great work!
U002	Simran	simran@email.com	AlphaBuild	Tester	Backend Dev	API Setup, DB Config	Innovator	U001	Very thorough!

3.1 Unnormalized Form

This structure violates atomicity and introduces redundancy.

3.3 First Normal Form (1NF)

To achieve 1NF, all attributes must contain atomic values. Multi-valued fields such as Subtasks, Badges_Earned, and Ratings are decomposed into individual rows.

User_ID	Full_Name	Email	Project_Name	Role_in_projects	Task_Name	Subtask_Title	Badge_Name	Rated_By	Teamwork	Communication	Feedback
U001	Raj	raj@email.com	Alpha Build	Developer	UI Design	Wireframe	Leader	U002	4	5	Great work!
U001	Raj	raj@email.com	Alpha Build	Developer	UI Design	Mock up	Contributor	U002	4	5	Great work!
U002	Simran	simran@email.com	Alpha Build	Tester	Backend Dev	API Setup	Innovator	U001	5	4	Very thorough!
U002	Simran	simran@email.com	Alpha Build	Tester	Backend Dev	DB Config	Innovator	U001	5	4	Very thorough!

3.2 1NF Table

All attributes now contain atomic values. However, redundancy and partial dependencies remain.

3.4 Second Normal Form (2NF)

To achieve 2NF, partial dependencies are removed. Attributes that depend only on part of a composite key are separated into distinct tables.

User_ID	Full_Name	Email	Bio	Skills	Profile_Pic	Points	Leadership_points	Password_hash
U001	Raj	raj@email.com	...	...	...	1200	Team Lead	...
U002	Simran	simran@email.com	...	...	...	950	Member	...

3.3 Users Table

Team_id	User_ID	Project_ID	Role_in_projects	Contribution_points
TM001	U001	P101	R01	Developer
TM002	U002	P101	R02	Tester

3.4 Project Members Table

Task_ID	Task_title	Description	Status	Deadline	Assigned_to	Kanban_column	Created_At	Project_ID
T001	UI Design	...	Open	2025-08-30	John	...	2025-07-25	P101

3.5 Tasks table

Subtask_ID	Subtask_title	Status	Task_ID
ST001	Wireframe	Done	T001

3.6 Subtasks Table

Badge_ID	Badge_Name	Description	Points_Required
B001	Leader	...	1000
B002	Contributor	...	500

3.7 Badges Table

User_ID	Badge_ID	Earned_At
U001	B001	2025-07-01
U001	B002	2025-07-15

3.8 BadgesEarned Table

rating_id	rated_by	rated_user	project_id	teamwork	communication	feedback
R001	U002	U001	P101	4	5	Great work!
R002	U001	U002	P101	5	4	Very thorough!

3.9 Ratings Table

Each non-key attribute is now fully functionally dependent on the primary key of its respective table.

3.4 Third Normal Form (3NF)

To achieve 3NF, transitive dependencies are eliminated.

- assigned_to in **Tasks** references user_id directly, avoiding transitive dependency
- rated_by and rated_user in **Ratings** are foreign keys pointing to **Users**, ensuring no transitive dependency

All non-key attributes are dependent only on the primary key of their respective tables.

3.5 Boyce-Codd Normal Form (BCNF)

BCNF resolves anomalies caused by dependencies on non-candidate keys.

In this schema:

- leadership_points is stored in the **Users** table and depends only on user_id, which is a candidate key
- All junction tables (e.g., **Project_Members**, **User_Badges**) are designed such that all dependencies are on candidate keys
- Ratings are structured so that each rating is uniquely identified by rating_id, and all dependencies are on candidate keys

No violations of BCNF exist in the current design.

3.6 Summary of Normalization

Normal Form	Objective	Achieved By
UNF	Raw data with redundancy	Initial combined table
1NF	Atomic values	Split multi-valued fields
2NF	Remove partial dependencies	Separate entities: Users, Projects, Tasks, etc.
3NF	Remove transitive dependencies	Isolate dependent attributes
BCNF	Eliminate non-key dependencies	Decompose based on candidate keys

Database Schema

A **Database Schema** is the logical blueprint of a database that defines how data is organized, stored, and related. It specifies the tables, attributes (columns), primary keys, foreign keys, and the relationships among entities, ensuring data integrity and consistency.

In our project, the schema has been designed for the **Collaborative Project Management System**, which manages users, projects, tasks, subtasks, team match requests, ratings, badges, and streaks. The schema is represented using an **Entity–Relationship (E-R) Diagram** that highlights:

- **Entities** like Users, Projects, Tasks, Subtasks, Ratings, and Badges.
- **Attributes** such as user details, project descriptions, deadlines, feedback, and achievements.
- **Relationships** including users joining projects, tasks belonging to projects, subtasks linked to tasks, and users earning ratings or badges.

This structured schema minimizes redundancy, maintains consistency, and ensures that the system effectively supports collaboration, progress tracking, and performance evaluation across multiple users and projects.

4.1 Users Table

```
CREATE TABLE Users (  
    user_id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    full_name VARCHAR(100) GENERATED ALWAYS AS (CONCAT(first_name, ' ',  
    last_name)) STORED,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    bio TEXT,  
    profile_pic VARCHAR(255),  
    points INT DEFAULT 0,  
    leadership_points INT DEFAULT 0,  
    skills JSON,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

4.2 Tasks Table

```
CREATE TABLE Tasks (  
    task_id INT AUTO_INCREMENT PRIMARY KEY,  
    task_title VARCHAR(255) NOT NULL,  
    description TEXT,  
    status ENUM('To Do', 'In Progress', 'Review', 'Done') DEFAULT 'To Do',  
    kanban_column VARCHAR(50),  
    deadline DATE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    assigned_to INT,  
    project_id INT,  
    FOREIGN KEY (assigned_to) REFERENCES Users(user_id),  
    FOREIGN KEY (project_id) REFERENCES Projects(project_id)  
);
```

4.3 Projects Table

```
CREATE TABLE Projects (  
    project_id INT AUTO_INCREMENT PRIMARY KEY,  
    project_name VARCHAR(255) NOT NULL,  
    description TEXT,  
    tech_stack VARCHAR(255),  
    workspace VARCHAR(255),  
    status ENUM('Active', 'Completed', 'On Hold') DEFAULT 'Active',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    closed_at TIMESTAMP NULL,  
    leader_id INT,  
    FOREIGN KEY (leader_id) REFERENCES Users(user_id)  
);
```

4.4 Project Members Table

```
CREATE TABLE Project_Members (  
    project_id INT,  
    user_id INT,  
    team_id INT,  
    role_in_projects VARCHAR(255),  
    contribution_points INT DEFAULT 0,  
    PRIMARY KEY (project_id, user_id),  
    FOREIGN KEY (project_id) REFERENCES Projects(project_id),  
    FOREIGN KEY (user_id) REFERENCES Users(user_id),  
    FOREIGN KEY (team_id) REFERENCES Project_Members(team_id)  
);
```

4.5 Team Match Requests Table

```
CREATE TABLE TeamMatchRequest (  
    request_id INT AUTO_INCREMENT PRIMARY KEY,  
    team_id INT,  
    user_id INT,  
    status ENUM('Pending', 'Accepted', 'Rejected') DEFAULT 'Pending',  
    requested_role VARCHAR(255),  
    FOREIGN KEY (team_id) REFERENCES Project_Members (team_id),  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

4.6 Task Streaks Table

```
CREATE TABLE Task_Streaks (  
    streak_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT,  
    current_streak INT DEFAULT 0,  
    longest_streak INT DEFAULT 0,  
    last_completed DATE,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

4.7 Subtasks Table

```
CREATE TABLE Subtasks (  
    subtask_id INT AUTO_INCREMENT PRIMARY KEY,  
    task_id INT,  
    subtask_title VARCHAR(255),  
    status ENUM('Pending', 'In Progress', 'Completed') DEFAULT 'Pending',  
    FOREIGN KEY (task_id) REFERENCES Tasks(task_id)  
);
```

4.8 Badges Table

```
CREATE TABLE Badges (  
    badge_id INT AUTO_INCREMENT PRIMARY KEY,  
    badge_name VARCHAR(255) NOT NULL,  
    description TEXT,  
    points_required INT NOT NULL  
);
```


4.9 User_Badges Table

```
CREATE TABLE User_Badges (  
    user_id INT,  
    badge_id INT,  
    earned_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (user_id, badge_id),  
    FOREIGN KEY (user_id) REFERENCES Users(user_id),  
    FOREIGN KEY (badge_id) REFERENCES Badges(badge_id)  
);
```

4.10 Ratings Table

```
CREATE TABLE Ratings (  
    rating_id INT PRIMARY KEY AUTO_INCREMENT,  
    project_id INT NOT NULL,  
    rated_by INT NOT NULL,  
    rated_user INT NOT NULL,  
    teamwork INT CHECK (teamwork BETWEEN 1 AND 5),  
    communication INT CHECK (communication BETWEEN 1 AND 5),  
    feedback TEXT,  
    FOREIGN KEY (project_id) REFERENCES Projects(project_id),  
    FOREIGN KEY (rated_by) REFERENCES Users(user_id),  
    FOREIGN KEY (rated_user) REFERENCES Users(user_id)  
);
```

4.11 Contribution_Heatmap Table

```
CREATE TABLE Contribution_Heatmap (  
    heatmap_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT NOT NULL,  
    contributions INT DEFAULT 0,  
    contribution_date DATE NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

References:

1. Miro.com
2. BridgeLabz
3. Eraser.io
4. Visual Paradigm - <https://guides.visual-paradigm.com/designing-a-library-system-from-erd-to-normalization-to-database-schema/>